

CrimGuard

How the whole thing works

An insider-threat detection platform built into a working file-sharing site.

Part 1 explains it with no technical knowledge assumed.

Part 2 explains exactly how each piece works.

Part 3 is a ready-made outline for the slide deck.

Contents

Part 1 - In plain English	2
What problem this solves	2
The two halves: Red and CrimGuard	2
The one idea everything hangs off	3
The two cases that define the product	3
What happens as someone gets riskier	6
The traps	7
What it never records	8
 Part 2 - In detail	 9
Architecture: what talks to what	9
The 100 variables	10
Where each number comes from	11
How the score is calculated	13
Roles, clearance and confidentiality	16
Risk limiting	17
Leaving: the departure gate	18
Downloading above your clearance	18
Decoys, canaries and honeytraps	19
Behavioural biometrics	19
The identity throttle	21
Shadow AI, sign-in cost, email reputation	21
How a new person gets access	22
What admins see	22
 Part 3 - For the slide deck	 23
The numbers worth quoting	23
A suggested 14-slide outline	24
The figures, ready to drop into slides	25
Lines that land	26
Questions to expect	26

In plain English

No technical knowledge assumed. If you only read one part, read this one.

What problem this solves

Most security spending goes on keeping strangers out: firewalls, passwords, locks on the door. That does nothing about the far more common and far more expensive problem, which is the person who is **already inside and already allowed in**. The employee who copies the customer list on their way to a competitor. The contractor who downloads the source code the week before their contract ends. They do not break in. They log in, the way they do every morning.

The hard part is that their activity looks exactly like work, because it *is* work, right up until it is not. A person reading 600 files might be doing a database migration or might be emptying the filing cabinet. Nothing about the number 600 tells you which.

The problem in one sentence

You cannot catch insider data theft by looking for unusual activity, because unusual activity is mostly just people doing their jobs. You have to look at whether there is a reason for it.

The two halves: Red and CrimGuard

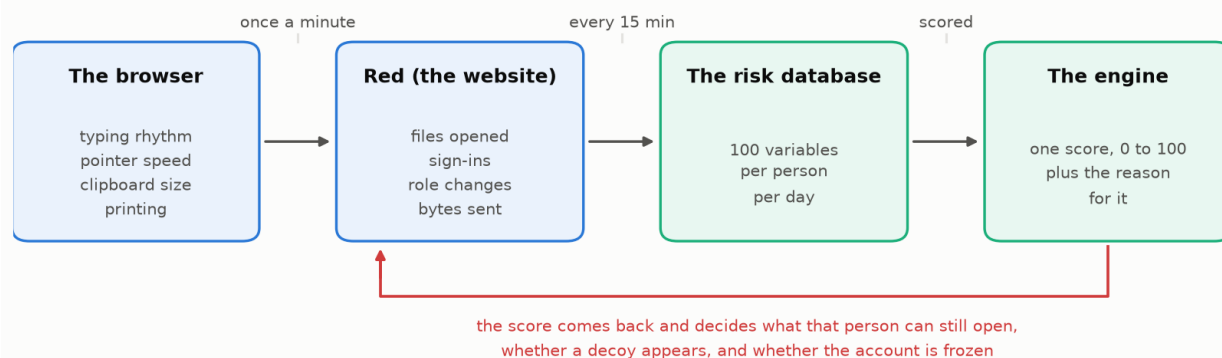
What we built is two things in one, and keeping them separate in your head makes the rest easy.

	Red	CrimGuard
What it is	A working file-sharing website. Projects, files, teams, logins, permissions.	A security system watching how everyone uses Red.
Who sees it	Everybody. It is the product people log into and do their work in.	Mostly invisible. Admins get a dashboard; everyone can see their own score.
What it does	Stores your projects and files. Controls who can open what.	Scores how each person behaves, and narrows or cuts their access when that score climbs.
Think of it as	The office building.	The security desk, the badge system and the cameras.

This matters because the detection is not a separate tool bolted on afterwards. It is built into the same site people work in, which is why it can see things a bolted-on tool cannot, and why it can act instantly rather than filing a report for someone to read on Monday.

How a click becomes a decision

Nothing here leaves the two databases. The whole loop closes in about fifteen minutes.



The whole loop. The browser and the website both feed the risk database; the engine turns that into one number, and the number comes back and changes what the person can do.

The one idea everything hangs off

Every account has a **risk score from 0 to 100**, recalculated continuously. It is built from **100 separate signals** about how that person behaves: how many files they open, when they work, how much they download, where they sign in from, even how they type.

Each signal is measured **against that person's own normal**, not against a company-wide rule. Someone who always works at midnight is not suspicious for working at midnight. Someone who never has, suddenly is.

Why "against their own normal" is the whole trick

A rule that says "alert if anyone downloads more than 500MB" fires constantly on the people whose job is moving data, and never fires on the person whose normal is 5MB and who just took 400. Comparing each person to themselves fixes both problems at once.

The two cases that define the product

These two situations are the reason the system is shaped the way it is. They are worth memorising, because they explain every design decision that follows.

	Case A - the false alarm	Case B - the real theft
What happens	An engineer reads 608 files in one day, up from a normal of 15.	An employee goes from 15 files a day to 70, gradually, over three months.
Is there a reason on file?	Yes - an approved migration ticket that expects about 600 files a day.	No ticket, no project, no role change. Nothing.
What a traditional tool does	Screams. It is a 40x spike. The analyst investigates, finds nothing, and trusts the tool a little less.	Nothing at all. No single day is unusual, so no threshold is ever crossed.
What CrimGuard does	Scores it 17.8 - low. The ticket explains it, and the amount is proportionate to what the ticket expects.	Scores it 72.9 - high. It compares against a frozen snapshot of months ago, not a rolling average that quietly absorbs the creep.

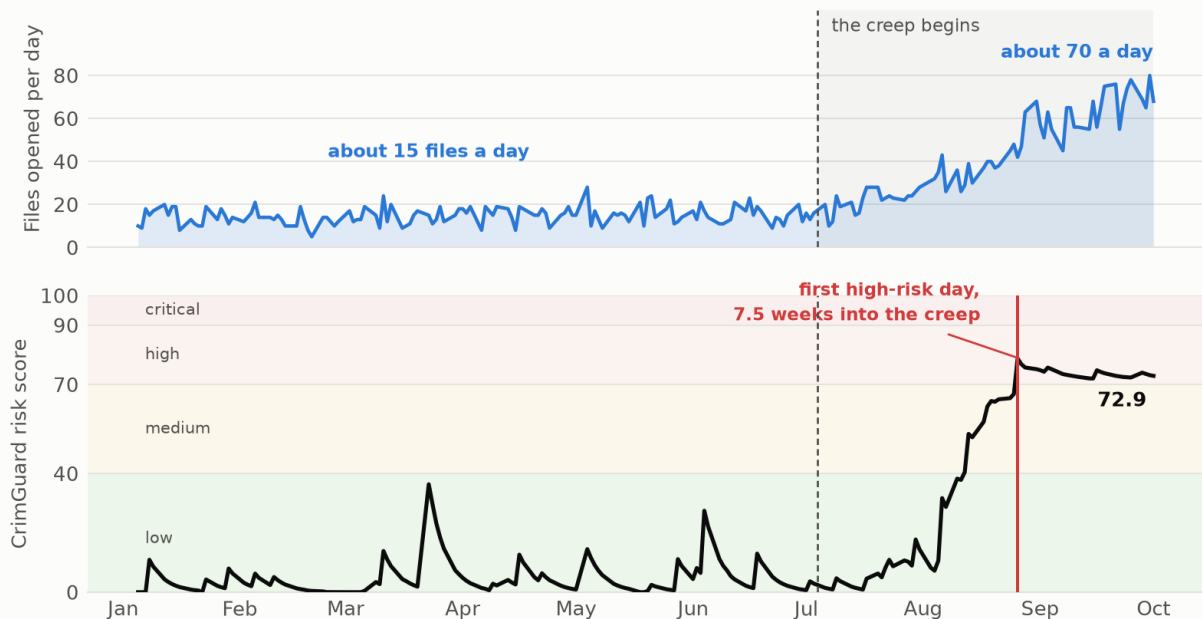
Say this in the presentation

Traditional monitoring gets both of these exactly backwards: it raises an alarm about the person doing their job, and stays silent about the person stealing. We flag the second and clear the first. And if that same engineer had done the identical thing with *no* ticket on file, the score goes from 17.8 to 96.3 - critical.

The reason Case B is invisible to normal tools is subtle and worth understanding. Most systems compare you to your recent average. If you creep up slowly, your recent average creeps up with you, so you never look unusual. CrimGuard also keeps an **anchored** comparison - a frozen picture of what you were like three to six months ago - which the creep cannot move. That is what catches it, roughly seven and a half weeks into the theft.

Case B - a slow theft the classic rule never sees

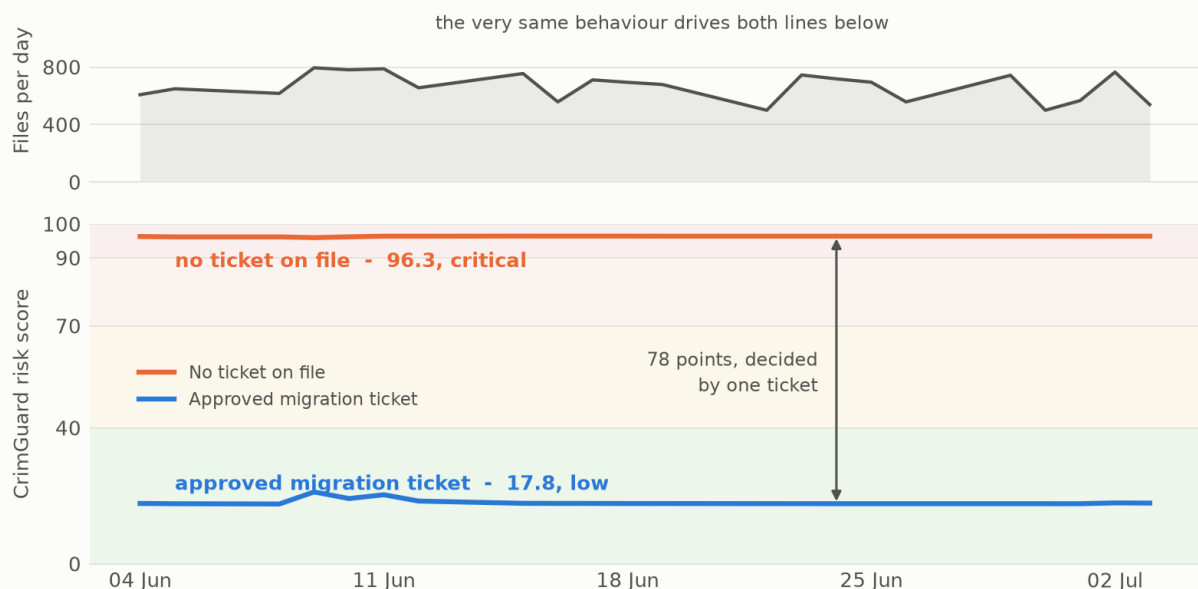
Fifteen files a day becomes seventy over three months. No single day is unusual, so a 3-sigma rule stays silent the whole way.



Case B, run through the real engine. The top panel is what the person did; the bottom is what CrimGuard made of it. Every point on both lines is real output, not an illustration.

Case A - the same 608-file day, with and without a reason on file

Context is the whole difference. A classic 3-sigma rule flags both of these identically.



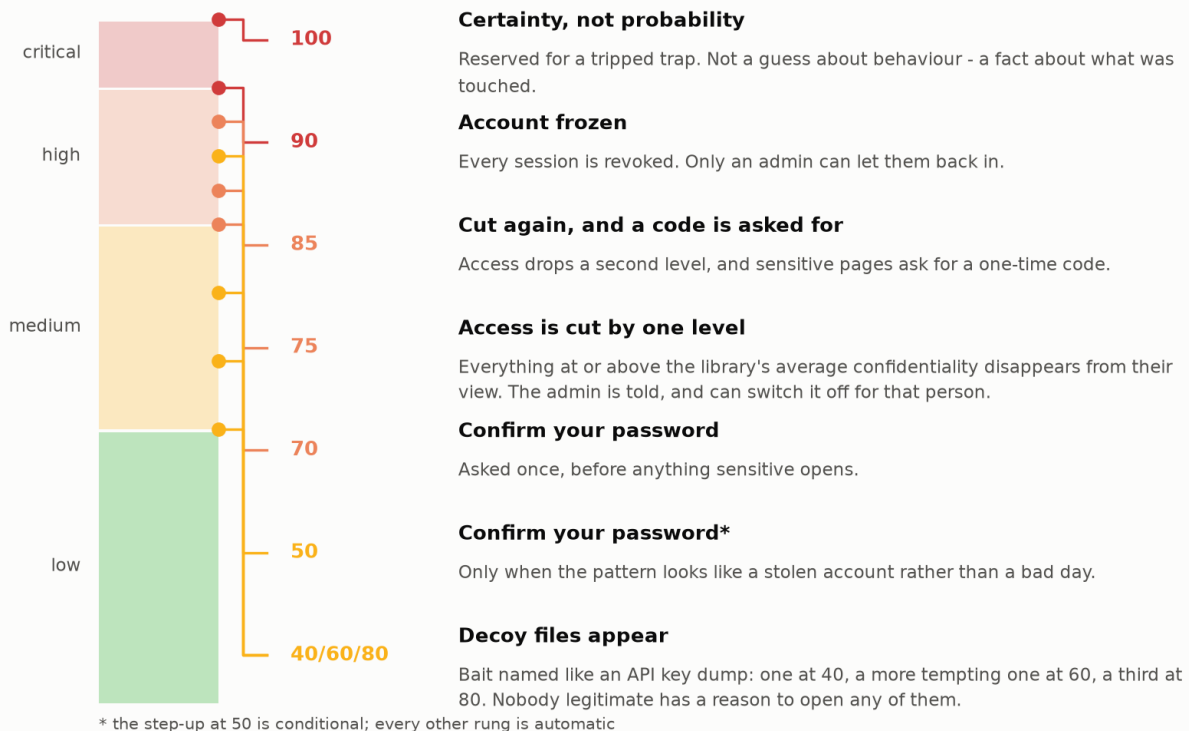
Case A, the same way. Both lines below come from identical behaviour: 608 files a day for a month. The only difference between them is whether a migration ticket exists.

What happens as someone gets riskier

The response is a ladder, not a switch. Nothing dramatic happens at a low score, and each step up is proportionate. All of it is automatic.

What happens as the score climbs

Nobody is locked out at the first sign of something odd. The response grows with the evidence.



Every rung, in order. A person on a bad day drifts into the 40s and meets a decoy they have no reason to touch. Losing access needs 75, and a freeze needs 90.

Score	What happens	Does the person notice?
40	A fake file appears among their files, designed to look like exactly what a data thief would reach for.	No. It looks like a real file.
50	If the pattern looks like a stolen account specifically, they are asked to confirm their password.	Yes - a password prompt.
60	A second, more tempting fake file appears.	No.
70	They must confirm their password before they can do anything else.	Yes.
75	Access narrows. Their clearance drops one level - files they could open yesterday stop opening. They also cannot upload.	Yes, clearly.
80	A third fake file appears.	No.
85	Access narrows again, by a second level. Separately, they are asked for a verification code.	Yes.
90	The account is frozen. Every session ends and they cannot sign back in until an admin lets them.	Yes. They are locked out.

Score	What happens	Does the person notice?
100	Reserved for certainty - see the traps below.	Locked out.

Two things about this ladder are deliberate. First, **the early steps are silent**: if someone is stealing, you do not want to tell them they have been noticed, you want to give them a marked banknote. Second, **an admin can switch the access-narrowing off** for a specific person if it is getting in the way of real work - but an admin cannot switch it off for themselves, or for another admin. Only the CEO can do that. Otherwise the control would be optional for exactly the people with the most access.

The traps

This is the part that tends to land best in a presentation, because it is the one place the system stops guessing and becomes certain.

Once someone looks risky, a **fake file appears that only they can see**. It is named like the thing a person stealing data would go for: "Production API keys (do not share)", "Payroll and compensation - all staff", "Customer master list". It is not real. Nothing legitimate ever needs to touch it.

If they...	What we conclude	What happens
Open it and look	Nothing. Curiosity is not theft, and the file is empty.	Nothing.
Try to change or delete it	They were acting on it, not just looking.	Every session ends instantly, the account is frozen, and the score is pinned at 100.
Copy the fake password inside it and paste it somewhere	Certainty.	The same, and we know where it went.
Try to use the fake password to log in somewhere	Certainty, plus whoever used it.	Both accounts frozen.

Why this is different from everything else in the system

Every other signal is probability - this behaviour is unusual, therefore it might be theft. A trap trip is not probability. Nobody touches a file that does not exist and was never mentioned by accident. It is the one signal that needs no context to interpret.

What it never records

Expect this question, because it is the first one a thoughtful person asks. The honest answer is good, and it should be said before anyone has to ask.

What we DO record	What we NEVER record
How fast you type - the average gap between keystrokes over a minute, and how long keys are held.	Which keys you pressed. Key identity never leaves your browser. Nothing you type is transmitted or stored.
How fast the pointer moves , as an average.	Where the pointer went. No positions, no paths, no clicks.
That you copied something , how many characters, and whether it looked like a password or a key.	What you copied. The text is checked inside the page and thrown away. Only the category name survives.
Which files and pages you opened , and when.	What is inside your files. The system records that a file was opened and how big it is, never its contents.
Sign-ins, the browser, roughly which network you came from.	Your precise location. The network your address belongs to and your browser time zone, nothing finer.

Two further points that matter. **Everyone can see their own score** and all 100 variables behind it, live, from a button in the corner of every page - because scoring people and hiding it from them would be worse. And there is a page at **/privacy** that says all of the above to the people being scored, in the same plain language.

The line to use if challenged

We record the rhythm of the typing, not the typing. It is the difference between recognising someone by their walk and reading their diary.

In detail

How each piece actually works. Technical, but written to be readable.

Architecture: what talks to what

There are two databases, deliberately separate.

	The website database	The risk database
Holds	Accounts, passwords, roles, projects, files, who can see what, the activity log.	The 100 variables per person per day, raw behaviour events, risk scores, alerts, the HR timeline.
Engine	SQLite, always.	PostgreSQL if configured, otherwise a SQLite copy.
Why separate	It is the product. It must work whether or not anyone is watching.	It is the detection platform, designed to also ingest badge readers, EDR and HR feeds that a website has nothing to do with.

Red runs perfectly without the risk database attached. Detach it and every page behaves identically; nothing is recorded and nothing is scored. That was a design rule, not an accident - the security layer is never allowed to be the reason the product breaks.

The flow, end to end

1	Somebody uses Red	They open a file, sign in, type, copy something, download an export.
2	It is recorded	The browser reports what only it can see (typing rhythm, clipboard size). The server records everything it can see for itself, where a page cannot forge it.
3	It is aggregated	Every 15 minutes, the raw events for each person become one daily snapshot of 100 numbers.
4	It is scored	The engine turns those 100 numbers into one score from 0 to 100, plus a reason for every point of it.
5	Something happens	The score is copied back to the website database, where it decides what that person can still open, whether a trap appears, and whether the account is frozen.

An important architectural detail worth a slide

Scores are calculated in the risk database, but access is decided in the website database. So each score is copied across as it is produced. That is why narrowing someone's access is instant and automatic rather than a nightly batch job.

The 100 variables

The catalogue of 100 risk variables is the backbone. Each one has a defined type, a direction that counts as suspicious, a weight, and a flag saying whether context can ever excuse it. They fall into ten groups.

Group	Filled	What it measures
Access and resources	10 / 10	Which projects, pages and account records were opened; what was searched for
Timing	10 / 10	When someone works, against their own pattern and the company's hours
Data movement	4 / 10	Exports, printing, clipboard size. No USB, personal cloud or email
Authentication and identity	6 / 10	Sign-ins, failures, new devices, new networks and countries, replayed session cookies
Device and network	5 / 10	Browser fingerprint, address ranges, impossible travel, bytes served
Behavioural biometrics	9 / 10	Typing and pointer rhythm, scrolling, idle time, focus changes, clipboard
HR and organisation	10 / 10	Role changes, plus employment details admins record (leaving date, reviews, leave)
Communication	2 / 10	Sensitive terms and secret-shaped patterns in the text people write
Privilege and permission	7 / 10	Admin actions, and refusals that show someone reaching past their role
Physical and environmental	1 / 10	Printing a page with confidential material on screen

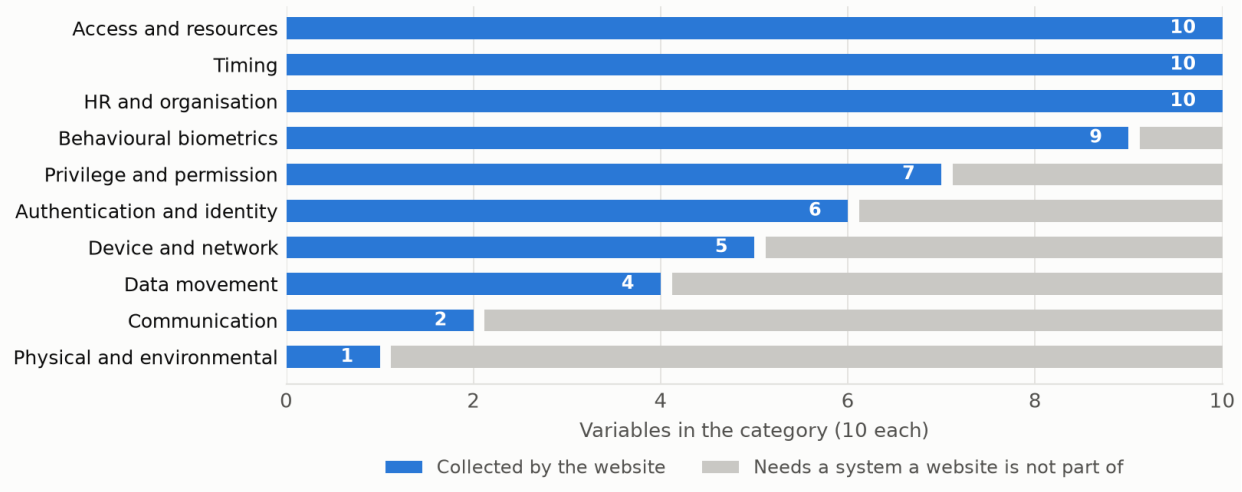
64 of 100 - and why the other 36 are blank, not zero

The missing 36 need systems a website is not part of: badge readers, an endpoint agent, device management, a mail gateway, a data-loss proxy. Those are written as **blank**, never as zero, because "we have no badge reader" and "nobody entered the building" are completely different facts. A file records the decision for every one of the 100 variables, and the system refuses to start if that file and the catalogue ever disagree.

That last point is worth making in the deck: it is an honesty mechanism. It is very easy for a dashboard to show a confident zero for something it has never measured. This one cannot.

64 of the 100 variables are filled by the website

The other 36 need badge readers, an endpoint agent, device management, a mail gateway or a DLP proxy. They are left blank, never written as zero.



The same table as a picture. Three categories are complete; the thin ones are thin because the missing signals live in hardware and gateways a website has no view of.

Where each number comes from

The split between what the browser reports and what the server records is a security decision, not a convenience one.

	The browser reports	The server records
What	Only what a server cannot possibly see: typing rhythm, pointer speed, scrolling, idle time, focus changes, clipboard size, printing, screen capture, and the browser's description of itself.	Everything it can see for itself: which files were opened, what was searched for, who signed in, who changed a role, how many bytes were sent.
Why	A page can lie about anything. So the browser is trusted with as little as possible, and nothing it sends can name another person - the account comes from the session.	A page cannot forge it. This is the trustworthy half.
How often	Batched once a minute; every 15 seconds while someone has their own risk panel open.	As it happens.

A worked example: impossible travel

This is a good one to demo because the arithmetic is obvious and the result is intuitive.

Two sign-ins are compared. Each is placed on the map, the straight-line distance is taken, and that is divided by the time between them. Over **900 km/h** across at least **400 km** is not a journey anybody made.

Two sign-ins	Distance	Implied speed	Verdict
Berlin, then Lagos 20 minutes later	5,196 km	15,589 km/h	Impossible
Paris, then Lagos 45 minutes later	4,709 km	6,279 km/h	Impossible (same time zone!)
Berlin, then New York 2 hours later	6,386 km	3,193 km/h	Impossible

Two sign-ins	Distance	Implied speed	Verdict
Berlin, then New York 10 hours later	6,386 km	639 km/h	A normal flight
Amsterdam, then Brussels 1 minute later	173 km	10,394 km/h	Too close to judge

The Paris-to-Lagos row is the one worth pointing at. Those two cities are in the **same time zone**, so a system comparing time-zone offsets would never see it. And the ten-hour New York row is the reverse mistake: an offset comparison would flag a perfectly normal flight every single time. Measuring actual distance and speed gets both right.

There is a guard as well: **the network has to have changed too**. A VPN moves your address without moving you, and changing your laptop's region setting moves neither. Neither trips this.

How the score is calculated

This is the most technical section. The short version for a slide is: **every signal is turned into a number between 0 and 1 saying how strong the evidence is, multiplied by how much that signal matters and how valuable the data was, reduced by how well it is explained, and then combined.**

Step 1 - Is this unusual for this person?

For each numeric signal, the last 30 days of that person's own history give a typical value and a spread. Robust statistics are used (median and median-absolute-deviation rather than mean and standard deviation) because the outliers we are hunting would otherwise drag the "normal" up towards themselves.

- A brand-new account has no history, so it is judged against **people in the same role** until it builds one - blending from peers to self over about ten days.
- A separate **anchored** comparison looks at a frozen window three to six months back, which is what catches slow creep (Case B).
- For yes/no signals, what matters is **how rare it is for that person**. A first-ever weekend login is strong evidence. For somebody who works most weekends it is worth nothing.

Step 2 - How much does this signal matter?

Not all signals are equal, and neither is all data.

Weight	Signals
Highest (1.0)	Data movement, and the strong precursors: compressing before a transfer, personal cloud, USB, audit-log tampering, impossible travel, session-token reuse, auto-forward rules, access granted with no ticket
High (0.8-0.9)	Access and privilege, authentication and device
Middle (0.5-0.7)	Communication, biometrics, physical, timing
Lowest (0.3-0.4)	Deliberately noisy ones - application switching, message volume

This is multiplied by **how valuable the data touched was**, on a 1-to-5 scale. The same behaviour against the payroll database and against a wiki page are not the same event.

Step 3 - Is there a reason for it?

This is the part that clears Case A, and the part most tools do not have at all. The system looks for a ledger entry that covers the behaviour - an assigned ticket, a project, a recent role change - and scores how well it explains what happened, out of four factors:

Factor	Question it answers
Reliability	Was the ticket approved? A project is weaker than an approved ticket; a role change weaker still.
Timing	Was the ticket open at the time? Something back-dated after the activity counts for nothing.
Scope	Were the files touched actually in the ticket's scope?
Proportionality	The ticket expects 600 files. They read 650, which is fine. They read 3,000, which is not.

A ticket is never a full excuse

Explanation is capped at 90%. A legitimate-looking reason can be cover, and the system is built so that filing a ticket can reduce your score but never zero it. Only the best single explanation counts, because two weak excuses do not add up to one good one.

Step 4 - Combine it all

- **Within a group**, signals overlap - a bulk copy raises the file count, the megabytes and the confidential-file count together - so the second strongest counts half, the third a quarter, and so on.
- **Across groups**, evidence is treated as independent and combined so that several moderate signals in different areas add up to a strong one.
- **Sequences** are looked for specifically: collect, then stage, then send. Two or three of those stages on the same day earns an extra penalty on top, because that ordering is what exfiltration actually looks like.
- **Over time**, each signal keeps its strongest recent value with a three-day half-life, so a sustained problem counts once rather than resetting every night.
- **Personal circumstances amplify but never create.** A resignation on file, a bad review, a disciplinary action, or booking most of the remaining holiday at once all multiply existing risk. If the behaviour score is zero, it stays zero.

The single most important sentence about the scoring

Every point of the final score can be traced back to the exact signal that produced it. The contributions add up to the total by construction - so the dashboard can always answer "why is this person at 84?" with a list, not a shrug. A black box nobody can interrogate would not survive contact with an actual investigation.

Does it work?

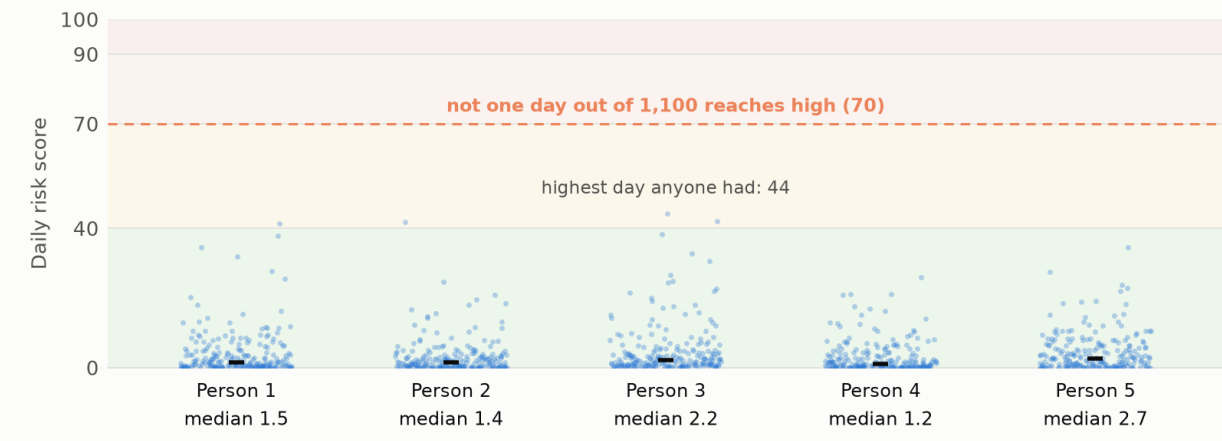
Tested against generated data with known answers:

Scenario	A classic 3-sigma tool	CrimGuard
Case A: 15 to 608 files/day, approved ticket on file	Flagged (wrongly)	17.8 - low , 90% explained
The same, with no ticket on file	Flagged	96.3 - critical
Case B: 15 to 70 files/day over 90 days, nothing on file	Missed entirely	72.9 - high , caught at ~7.5 weeks
5 ordinary people over ~220 working days each	-	Median 1-3, zero high-risk days

That last row is the one to keep in your back pocket. A detector that catches everything by crying wolf is useless; the false-positive rate is the number that decides whether anyone keeps using it.

Five ordinary people, about 220 working days each

Every dot is one person on one day. The false-alarm rate decides whether anyone keeps using a detector at all.



Every scored day for the five ordinary people. Five clusters, about 1,100 days, and not one of them crosses into high. The highest single day anyone had was 44.

Roles, clearance and confidentiality

Two matching 1-to-5 scales. Roles have **clearance**; files have **confidentiality**. Your clearance must reach a file's confidentiality.

Role	Clearance	File level	Name
Intern	1	1	Open
Employee	2	2	Internal (the default for new files)
Admin	4	3	Confidential
CEO	5	4	Restricted
(the CEO can create more)	1-5	5	Secret - CEO only

Who can open a file

- Its **owner** always can, and so can the **CEO**. Admins can open anything up to Restricted.
- Shared with a **role**: everyone in it can open the file, as long as the role's clearance covers the file's level. A Confidential file shared with Employees stays shut to them.
- Shared with a **person by name**: they can open it at any level. This is the deliberate exception, and it is the one the departure gate later closes.
- A file you cannot see returns "not found", so its existence is not given away.
- **Only the CEO can mark a file Secret**, or change who can see one.

One rule, written once

Who can see which file is expressed as a single database rule that every list, every download, every dialog and the whole dashboard goes through. There is no second copy that can drift out of step - which is exactly how permission bugs normally happen.

Risk limiting

The automatic narrowing of access. The baseline is the average confidentiality of every file in the system, rounded down.

Score	Clearance is capped at	If the average file is level 3
75 and over	one level below the average	held to level 2
85 and over	two levels below	held to level 1

Only clearance is cut. Owning a file, and having one handed to you by name, are not clearance decisions and are deliberately left alone. The point is to narrow how *wide* someone's reach is, not to lock them out of their own work mid-sentence.

Governance rule	Why
An admin can waive it for an ordinary person, with a reason.	Real work sometimes needs it, and the reason is logged.
An admin cannot waive it for themselves.	Otherwise the control is optional for the people with the most access.
An admin cannot waive it for another admin.	Same reason. Only the CEO can.

Leaving: the departure gate

Risk limiting only cuts clearance, and a file handed to you **by name** was never a clearance decision - so limiting leaves it alone. That is the right default, until somebody is leaving. Insider IP theft clusters in the weeks around departure, and this is the one door limiting does not close.

So inside the last **14 days** before a leaving date, and only there:

The person	The file	What happens
Leaving in 14 days or fewer	Handed to them by name, above their clearance	Refused, with a request to make
Leaving	Within their clearance, or their own	Opens as it always did
Not leaving	Anything	Opens as it always did

- **The file is still listed.** Refusing is not hiding - they have to see it to ask for it, and the refusal carries the request rather than dead-ending.
- The request lands in an admin queue with the file, its level, who is asking, their reason and how many days they have left.
- **Approving takes the clearance the file needs.** An admin cannot approve their way into a Secret file.
- An approval is **a key to one file and expires after 7 days**, so nobody has to remember to take it back.
- **Reaching for a held file is recorded** - in the activity log and as a signal the risk engine sees. It is the only trace that would otherwise exist of someone trying on the way out.

Downloading above your clearance

A file handed to you by name, or released by an admin, can sit above your clearance. Two rules apply when you take a copy of one.

Rule	How it works
Every download asks for a code.	The download is refused until a verification code is entered. One code is good for one download of that file, on that session, within 2 minutes. Five wrong codes shut that file on that session. Passes, failures and lockouts are all logged.
Grabbing it instantly raises the score.	A download attempted within 5 seconds of being given the file adds +20 to the score for 7 days . It is judged on the first attempt, before the code is asked for, so entering the code cannot hide it. A retry is the same grab, not a second one.

The reasoning behind the second rule is worth saying aloud: somebody who is sent a file and opens it five seconds later was waiting for it. That is not how people work through a shared document; it is how people collect one.

Decoys, canaries and honeytraps

Two layers, at two different levels of the system.

Layer 1 - decoy projects

At score **40**, a fake project appears in that account's list. Only they can see it. Opening it does nothing.

Changing or deleting it is the trip: every session ends, the account is frozen, the next score is pinned at 100, and a report prints to the server console naming the person, their score at that moment, which decoy and what they did.

Layer 2 - decoy files with live canaries

More elaborate. The fake file contains **freshly minted fake credentials** - things that look exactly like real API keys and passwords. Those are watched for wherever they reappear:

Where the fake credential turns up	What it proves
On the clipboard, inside our own pages	They copied it.
Written into text somewhere else in the system	They moved it.
Presented as a login to one of our trap endpoints	They tried to use it - and we learn who did.

A trip freezes **everyone involved**: the person the decoy was planted for, and whoever presented the credential. The credentials stay armed even after the decoy is removed, so a copy taken today still reports itself when it is used next month.

Why traps are worth a slide of their own

Every other signal is an inference. A trap trip is not. This is the difference between "this person is behaving unusually" and "this person took something that only exists to be taken" - and it is the one piece of evidence that survives an argument.

Behavioural biometrics

The system learns how each person types and moves the mouse, and notices when that changes. Answering, in effect: **is the account owner actually the one at the keyboard?**

Step	What happens
1. Build a profile	From the owner's own accepted sessions, each measurement gets a typical value and a spread.
2. Compare	A new minute of activity is compared feature by feature. Each feature is capped so one odd measurement cannot decide alone.
3. Calibrate	The same comparison is run on all the owner's own past sessions, so the question becomes "how far outside their own variation is this?" - which makes a steady typist and an erratic one comparable.
4. Combine	Typing and pointer evidence are merged so that two moderate signals that agree add up to a strong one.

Crucially this is judged **together with the risk score**, because each means something different alone:

Typing has changed	Risk score	Conclusion
Yes	Normal	Probably a different person at the keyboard. Ask them to confirm their password.
Somewhat	Raised	An anomaly <i>alongside</i> a physical change. Ask as well.
No	Raised	It is them, and they are doing something unusual. The other controls handle it.

The identity throttle

One place where everything that acts on an account comes together: a score crossing a policy, a change in typing, a trap trip, failed confirmations. Three actions are possible.

Action	What it does	Triggered by
Step-up	The session can reach nothing but the confirmation prompt until the owner confirms with their password.	Score 70; score 50 if it looks like a stolen account; typing changed
Freeze	Every session ends and signing in is refused until an admin restores access.	Score 90; any trap trip
Revoke	Every session ends, but they can sign in again.	Lesser events

- Every action is written to a response log, so state survives a restart and an admin can see exactly what was done and why.
- A policy can be set to **require an admin's approval** before it acts, rather than acting immediately.
- **Two challenges can be open at once** - a score of 87 crosses both the password policy at 70 and the code policy at 85. Both stay answerable, because two gates that each only allowed their own answer through would block each other and leave the person with nothing to do but sign out.
- Passing the code challenge earns a **bounded, expiring discount of 25 points for 12 hours**. Proving who you are answers the most likely innocent explanation for a high score - somebody else is on this session - but it does not undo what the detectors actually saw, so it can never take an account below the medium band on its own.

Shadow AI, sign-in cost, email reputation

Shadow AI

Work leaving the company into an AI model nobody approved. The module is explicit about the limits: a website can see, with certainty, **that a selection was copied out of one of its own documents**, how large it was, which file and project it came from, how confidential that was, and whether it looked secret-shaped. It cannot see another browser tab, another application, or an extension - and no amount of JavaScript changes that. The detection is built on what can actually be known.

A cost on every sign-in attempt

Before any password is checked, the browser must do a small amount of real computation. This is not a CAPTCHA and does not try to tell a human from a machine - a model that can read distorted text reads it better than the person it was meant to admit. What it does is make each attempt **cost CPU time**, which is the thing large-scale credential stuffing depends on being free. It turns thousands of guesses a second into a few, and costs an honest person a few hundred milliseconds they never notice.

Email reputation

Whether an address appears in known breach data, or belongs to a throwaway domain. Small numbers on purpose: neither is evidence of wrongdoing, so like the HR factors, it can raise existing risk but never invent it.

How a new person gets access

Two ways access arrives without anyone granting it file by file.

How it works	
A project shared with a role	Everyone in that role can see the project's files, including ones uploaded later - which is the whole reason to share a project rather than a file. Clearance still applies: sharing a project with the interns does not hand them a Secret file inside it.
A new account provisioned from its peers	A new intern should not begin with nothing and wait for somebody to notice. The system looks at the people who already hold that role, counts how many files each was given, and hands the newcomer the same number - taking the files the most role-mates already have. Bounded twice: never above the role's clearance, and never at all if there are no peers to copy.

What admins see

Screen	What is on it
CrimGuard dashboard	Everyone with an account: role, projects, files, storage, whether they are online, and their risk score. Choose someone and their full record opens - every project and file with its sharing, files shared with them, the devices they are signed in on, their activity and their risk trend. It refreshes itself while open.
Risk console	Everyone's score, open alerts, and one person's full list of 100 variables with each one's contribution to the score. Also where the HR context is recorded.
Activity log	Sign-ins, role and password changes, changes to who can see a file, downloads of other people's files, and records opened in CrimGuard. Streams into a live console view.
Access requests	The queue of people asking for a file the departure gate is holding.
Everyone's own panel	A button in the corner of every page showing that person their own score and all 100 variables, live.

Two restraints worth mentioning

File names above the viewer's clearance are **counted but never shown**, so an admin can see that somebody holds Secret files without learning what they are. And **looking is itself recorded** - opening someone's record goes in the activity log. A system holding everyone's risk record has to watch its own watchers.

For the slide deck

Numbers, an outline, and the questions to expect.

The numbers worth quoting

100	risk variables in the catalogue
64	of them filled by the website; the other 36 need hardware or systems a website is not part of
10	categories, from file access to typing rhythm to HR context
0-100	the risk score, recalculated every 15 minutes and on demand
17.8 vs 96.3	the same 608-file day, with and without a ticket on file
72.9	the slow-creep theft that a 3-sigma rule scores 0.5 and ignores completely
7.5 weeks	how far into a three-month slow theft it is caught
0	high-risk days across five ordinary users over ~220 working days each
900 km/h	the impossible-travel threshold
75 / 85 / 90	access narrowed, narrowed again, account frozen
14 days	the departure window before someone leaves
270	automated tests, all passing
0	third-party packages - no npm dependencies at all

That last one is worth a mention. The whole thing runs on what is built into the language, which means nothing in the supply chain to compromise and nothing to keep patching - in a security product, that is a claim worth making.

A suggested 14-slide outline

#	Slide	The one thing it says
1	Title	CrimGuard - catching the theft that comes from inside
2	The problem	Security spending keeps strangers out. The expensive problem is the person already inside and already allowed in.
3	Why it is hard	Their activity looks exactly like work, because it is work - right up until it is not.
4	Case A and Case B	Traditional tools flag the engineer doing their job and miss the person stealing. Show both columns. <i>This is your strongest slide.</i>
5	The idea	One score, 0-100, from 100 signals, each measured against that person's own normal.
6	What we measure	The ten categories. Mention 64 of 100, and that the rest are honestly blank rather than fake zeros.
7	Context is the product	The four questions asked of any excuse: approved? in time? in scope? proportionate? Capped at 90% - a ticket can be cover.
8	The ladder	The escalation table: 40 bait, 70 confirm, 75 narrow, 85 narrow again, 90 freeze. Automatic and proportionate.
9	The traps	A fake file only they can see. Touch it and you are frozen. The one signal that is certainty, not probability.
10	Knowing it is really them	Typing and pointer rhythm, judged together with the score. Three rows of the truth table.
11	Leaving	Theft clusters around departure. The 14-day gate, and the request queue.
12	Privacy	The two-column do/do-not table. "The rhythm of the typing, not the typing." Everyone can see their own score.
13	Does it work?	The results table, and especially the zero false alarms over 1,100 ordinary user-days.
14	Close	Every point of every score traces back to the signal that produced it.

The figures, ready to drop into slides

Every chart in this document is saved as a PNG in docs/figures/ in the repository, at presentation resolution. Drag them straight onto a slide - none of them needs a caption beyond the title already printed on the image.

File	Use it on	What to say over it
case-b.png	Slide 4	Point at the top line first: nothing here looks wrong on any single day. Then the bottom line: the score climbs anyway.
case-a.png	Slide 4 or 7	Both lines are the same person doing the same thing. One ticket is the whole difference between 17.8 and 96.3.
ladder.png	Slide 8	Read it bottom to top. Nobody loses anything until 75.
coverage.png	Slide 6	64 filled, 36 honestly blank. The blanks are a feature.
false-positives.png	Slide 13	About 1,100 ordinary working days. Not one alarm.
architecture.png	Slide 5	One loop: watch, score, act. It closes in about fifteen minutes.

The charts were generated by running the risk engine itself, not drawn by hand. If the engine changes, re-running the chart script redraws them from the new output.

Lines that land

- "We are not trying to keep people out. Everyone here is already allowed in. That is the problem."
- "A 40x spike with an approved ticket behind it scores 17.8. The same spike with nothing behind it scores 96.3. The behaviour is identical - the context is not."
- "The theft that actually happens is not a spike. It is a creep. Fifteen files a day becomes seventy over three months, and no single day is ever unusual."
- "We record the rhythm of the typing, not the typing."
- "Nobody touches a file that does not exist and was never mentioned to them by accident."
- "Every point of the score can be traced to the signal that produced it. If you cannot explain a score, you cannot act on it."
- "An admin can turn the restriction off for you. They cannot turn it off for themselves."

Questions to expect

Question	Answer
"Is this spyware?"	It records rhythm and counts, never content. No keystrokes, no pointer paths, no file contents, no clipboard text. Everyone can read their own full record at any time, and there is a page explaining all of it to them. Detach one database and nothing is recorded at all.
"What if it is wrong about someone?"	Nothing irreversible happens automatically. The early steps are silent, the middle ones ask you to confirm who you are, and an admin can waive the restriction with a reason. A freeze needs an admin to lift it - which means a human looks at it.
"Could an admin abuse this?"	An admin cannot waive their own restriction or another admin's - only the CEO can. Looking at someone's record is itself logged. File names above your clearance are counted but never shown to you.
"What stops someone gaming it?"	The browser is trusted with as little as possible, and nothing it sends can name another person. Everything the server can observe for itself is recorded server-side where a page cannot forge it. And a ticket never explains more than 90% of anything.
"Why not buy this?"	Commercial tools are built around one signal each, they alert rather than act, and they mostly cannot see context. This scores 100 signals together, weighs them against the reason on file, and narrows access by itself rather than emailing somebody.
"How do you know it works?"	Both cases from the pitch reproduce on demand, 270 automated tests pass, and across five ordinary simulated users over roughly 220 working days each there were zero high-risk days.

This document was generated from the working code, not from a plan. Every number in it was taken from the running system.